

My Project

Generated by Doxygen 1.13.2

1 README	1
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 Studentas Class Reference	11
5.2 Timer Class Reference	12
5.3 Zmogus Class Reference	12
6 File Documentation	13
6.1 humanClass.h	13
6.2 studentClass.h	13
6.3 timer.h	15
Index	17

Chapter 1

README

v2.0

v2.0 sukurta dokumentacija ir atlikti unit testai

v1.5 šakoje pridėta nauja klasė - Žmogus. Studento klasė dabar paveldi Žmogaus klasę, todėl atitinkamai buvo pakeisti ir konstruktoriai / destruktoriai. Nauja Žmogaus bei Studento klasė ištestuota su v1.2 versijoje sukurtu testavimo metodu. Rezultatai parodė, jog nauja klasė veikia su anksčiau implementuotais metodais, t.y. testo rezultatai sutampa su žemiau 1.2 versijoje aprašytais to pačio testo rezultatais.

Žemiau pridedamas testo kodas, leidžiantis geriau suprasti testo rezultatus:

```
cout << "Pasirinkote testuoti klasę: " << endl;
cout << "---- Default konstruktorius ----" << endl;
Studentas s1;
cout << "Egzaminas: " << s1.egzaminas() << endl;
cout << endl;
cout << "---- Vardo, pavardės konstruktorius ----" << endl;
Studentas s2("Jonas", "Jonaitis");
cout << "Vardas: " << s2.vardas() << ", pavardė: " << s2.pavarde() << endl;
cout << endl;
cout << "---- Pilnas konstruktorius ----" << endl;
vector<int> pazymiai = {8, 9, 10, 7, 8};
Studentas s3("Petras", "Petraitis", pazymiai, 9);
cout << "Vardas: " << s3.vardas()
<< ", pavardė: " << s3.pavarde()
<< ", egzaminas: " << s3.egzaminas()
<< ", galutinis vid: " << s3.galutinisVid()
<< ", galutinis med: " << s3.galutinisMed() << endl;
cout << endl;
cout << "---- Copy konstruktorius ----" << endl;
Studentas s4(s3);
cout << "s3 kopija - vardas: " << s4.vardas()
<< ", pavardė: " << s4.pavarde()
<< ", egzaminas: " << s4.egzaminas()
<< ", galutinis vid: " << s4.galutinisVid() << endl;
cout << endl;
s4.setVardas("Antanas");
cout << "Po kopijos pakeitimo - s3.vardas: " << s3.vardas()
<< ", s4.vardas: " << s4.vardas() << endl;
cout << endl;
cout << "---- Move konstruktorius ----" << endl;
Studentas s5(std::move(s4));
cout << "Movint'as iš s4 - vardas: " << s5.vardas()
<< ", pavardė: " << s5.pavarde()
<< ", egzaminas: " << s5.egzaminas() << endl;
cout << endl;
cout << "---- Copy assignment operatorius ----" << endl;
Studentas s6;
s6 = s3;
cout << "s6 po s3 kopijavimo - vardas: " << s6.vardas()
<< ", pavardė: " << s6.pavarde() << endl;
cout << endl;
cout << "---- Move assignment operatorius ----" << endl;
Studentas s7;
s7 = std::move(s6);
```

```

cout << "s7 po movinimo iš s6 - vardas: " << s7.vardas()
<< ", pavardė: " << s7.pavarde() << endl;
cout << endl;
cout << "---- Output'o operatorius ----" << endl;
cout << "s3 naudojant operatorių << " << s3 << endl;
cout << endl;
cout << "---- Input'o operatorius ----" << endl;
string testInput = "Tomas Tomauskas 5 8 9 10 7 9";
istringstream iss(testInput);
Studentas s8;
iss >> s8;
cout << "s8 po input: " << s8 << endl;
cout << endl;
cout << "---- Destruktorius ----" << endl;
testuotiDestruktoriu();
cout << "---- Žmogaus konstruktoriaus ----" << endl;
Zmogus z("Pranas", "Pranaitis");
cout << z.vardas() << " " << z.pavarde() << endl;

```

Kaip matoma iš žemiau pridėtos ekrano nuotraukos, testo rezultatai nuo v1.2 nepasikeitė:

Ankstesnėje v1.2 šakoje implementuoti ir testuoti "Rule of five" metodai bei perdengti įvesties ir išvesties operatoriai. Jų veikimas pritaikytas anksčiau naudotose programos dalyse, tokiose kaip: nuskaitymas iš failo, studentų įvedimas ranka, studentų rezultatų išvedimas.

Žemiau pateikiamos kodo iškarpos, rodančios "Rule of five" bei operatorių perdengimą.

```

//copy konstruktorius
Studentas(const Studentas &s)
: vardas_(s.vardas_),
  pavarde_(s.pavarde_),
  egzaminas_(s.egzaminas_),
  galutinisMed_(s.galutinisMed_),
  galutinisVid_(s.galutinisVid_),
  pazymiuVidurkis_(s.pazymiuVidurkis_),
  pazymiai_(s.pazymiai_){}

//move konstruktorius
Studentas(Studentas&& s)
: vardas_(std::move(s.vardas_)),
  pavarde_(std::move(s.pavarde_)),
  egzaminas_(s.egzaminas_),
  galutinisMed_(s.galutinisMed_),
  galutinisVid_(s.galutinisVid_),
  pazymiuVidurkis_(s.pazymiuVidurkis_),
  pazymiai_(std::move(s.pazymiai_)){}

//copy assignment operatorius
Studentas& operator=(const Studentas& s){
    if (this == &s) return *this;

    vardas_ = s.vardas_;
    pavarde_ = s.pavarde_;
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = s.pazymiai_;

    return *this;
}

//move assignment operatorius
Studentas& operator =(Studentas&& s){
    if (this == &s) return *this;

    vardas_ = std::move(s.vardas_);
    pavarde_ = std::move(s.pavarde_);
    egzaminas_ = s.egzaminas_;
    galutinisMed_ = s.galutinisMed_;
    galutinisVid_ = s.galutinisVid_;
    pazymiuVidurkis_ = s.pazymiuVidurkis_;
    pazymiai_ = std::move(s.pazymiai_);

    return *this;
}

//destruktorius
~Studentas(){
    vardas_.clear();
    pavarde_.clear();
    pazymiai_.clear();
}

```

```

};

//perdengtas įvesties operatorius (panaikintas vardo ir pavardės tikrinimas dėl failuose esančio
//formatavimo, kur vardas ir pavardė GALI turėti skaičius)
std::istream& operator>(std::istream& cin, Studentas& s){
    string vardas, pavarde;
    vector<int> pazymiai;
    int egzaminas, pazymys;

    cin >> vardas >> pavarde;

    if (cin.eof()) {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }

    while (cin >> pazymys){
        if (cin.fail()){
            if (cin.eof()){
                break;
            }
            cin.clear();
            cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra skaitinės reikšmės");
        }
        if (pazymys < 1 || pazymys > 10){
            throw std::runtime_error("Netinkamas failo formatas: pažymiai nėra sveiki skaičiai
ribose nuo 1 iki 10");
        }
        pazymiai.push_back(pazymys);
    }
    if (!pazymiai.empty()) {
        egzaminas = pazymiai.back();
        pazymiai.pop_back();
    } else {
        throw std::runtime_error("Netinkamas failo formatas: faile nėra pažymių");
    }
    s = Studentas(vardas, pavarde, pazymiai, egzaminas);
    return cin;
}

//perdengtas išvesties operatorius
std::ostream& operator<(std::ostream& out, const Studentas &s) {
    out << std::left << std::setw(20) << s.pavarde() << std::setw(20) << s.vardas() << std::setw(20) <<
    std::fixed << std::setprecision(2) << s.galutinisVid() << std::setw(20) << std::fixed <<
    std::setprecision(2) << s.galutinisMed() << endl;
    return out;
}

```

v1.2 versijoje sukurta klasės testavimo programa leidžia testuoti visas minėtas funkcijas, gaunami rezultatai:

```

---- Default konstruktorius ----
Egzaminas: 0

---- Vardo, pavardės konstruktorius ----
Vardas: Jonas, pavardė: Jonaitis

---- Pilnas konstruktorius ----
Vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76, galutinis med: 8.6

---- Copy konstruktorius ----
s3 kopija - vardas: Petras, pavardė: Petraitis, egzaminas: 9, galutinis vid: 8.76

Po kopijos pakeitimo - s3.vardas: Petras, s4.vardas: Antanas

---- Move konstruktorius ----
Movint'as iš s4 - vardas: Antanas, pavardė: Petraitis, egzaminas: 9

---- Copy assignment operatorius ----
s6 po s3 kopijavimo - vardas: Petras, pavardė: Petraitis

---- Move assignment operatorius ----
s7 po movinimo iš s6 - vardas: Petras, pavardė: Petraitis

---- Output'o operatorius ----
s3 naudojant operatorių <<: Petraitis          Petras          8.76          8.60

---- Input'o operatorius ----
s8 po input: Tomauskas          Tomas          8.52          8.60

---- Destruktorius ----
Destruktorius suveikė

```

Ankstesnė v1.1 šaka skirta palyginti "class" ir "struct" naudojimą talpinant studento duomenis. Tyrimas ir jo rezultatai pateikiami žemiau, v1.1 implementuota "class" lyginama su v1.0 versijoje naudotu "struct".

Tyrimas vykdytas su 3 strategija (žr. v1.0) bei vektoriaus kontaineriu, vykdytos 3 iteracijos.

"Class" ir "struct" | Tyrimo rezultatai

Programos veikimo laikas su CLASS	Programos veikimo laikas su STRUCT	Studentų kiekis	Optimizavimo vėliava	.exe failo dydis su CLASS (KB)	.exe failo dydis su STRUCT
0.434467s	0.418547s	100 000	-O3	332	329
0.453639s	0.428051s	100 000	-O2	319	315
0.453065s	0.427737s	100 000	-O1	317	331
1.54309s	1.32805s	100 000	-	861	841
1.95142s	1.70978s	1 000 000	-O3	332	329
1.9155s	1.73843s	1 000 000	-O2	319	315
1.93125s	1.7828s	1 000 000	-O1	317	331
7.94422s	6.41446s	1 000 000	-	861	841

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Timer	12
Zmogus	12
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	11
Timer	12
Zmogus	12

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

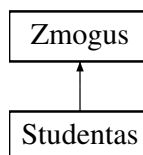
humanClass.h	13
studentClass.h	13
timer.h	15

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (string var, string pav, vector< int > paz, int egz)
- **Studentas** (string var, string pav)
- **Studentas** (std::istream &is)
- **Studentas** (const [Studentas](#) &s)
- **Studentas** ([Studentas](#) &&s)
- double **galutinisMed** () const
- double **galutinisVid** () const
- const vector< int > & **pazymiai** () const
- int **egzaminas** () const
- void **setGalutinisV** (double galutVid)
- void **setGalutinisM** (double galutMed)
- void **setEgzaminas** (int egz)
- void **setPazymiai** (vector< int > paz)
- void **skaiciuotiGalutiniSuVid** ()
- void **skaiciuotiGalutiniSuMed** ()
- [Studentas](#) & **operator=** (const [Studentas](#) &s)
- [Studentas](#) & **operator=** ([Studentas](#) &&s)

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (string var, string pav)
- std::string **vardas** () const
- std::string **pavarde** () const
- void **setVardas** (string var)
- void **setPavarde** (string pav)

Friends

- `std::ostream & operator<< (std::ostream &out, const Studentas &s)`
- `std::istream & operator>> (std::istream &cin, Studentas &s)`

The documentation for this class was generated from the following files:

- `studentClass.h`
- `programaFunkcijos.cpp`

5.2 Timer Class Reference

Public Member Functions

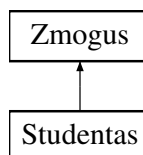
- `void reset ()`
- `double elapsed () const`

The documentation for this class was generated from the following file:

- `timer.h`

5.3 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- `Zmogus (string var, string pav)`
- `std::string vardas () const`
- `std::string pavarde () const`
- `void setVardas (string var)`
- `void setPavarde (string pav)`

The documentation for this class was generated from the following file:

- `humanClass.h`

Chapter 6

File Documentation

6.1 humanClass.h

```
00001 #include <iostream>
00002 using std::string;
00003
00004 class Zmogus {
00005     private:
00006         string vardas_;
00007         string pavarde_;
00008     public:
00009         Zmogus() {}
00010         Zmogus(string var, string pav) : vardas_{var}, pavarde_{pav} {}
00011         //get'eriai
00012         inline std::string vardas() const { return vardas_; }
00013         inline std::string pavarde() const { return pavarde_; }
00014         //set'eriai
00015         inline void setVardas(string var) { vardas_ = var; }
00016         inline void setPavarde(string pav) { pavarde_ = pav; }
00017
00018         virtual ~Zmogus() {
00019             vardas_ = "";
00020             pavarde_ = "";
00021         }
00022 };
```

6.2 studentClass.h

```
00001 #ifndef MANO_LIB_H
00002 #define MANO_LIB_H
00003
00004 #include <iostream>
00005 #include <iomanip>
00006 #include <vector>
00007 #include <fstream>
00008 #include <string>
00009 #include <numeric>
00010 #include <ctime>
00011 #include <sstream>
00012 #include <chrono>
00013 #include <unordered_set>
00014 #include <limits>
00015 #include <ios>
00016 #include "timer.h"
00017 #include "humanClass.h"
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::string;
00022 using std::vector;
00023 using std::endl;
00024 using std::accumulate;
00025 using std::ifstream;
00026 using std::istringstream;
00027
00028 class Studentas : public Zmogus {
```

```

00029     private:
00030         int egzaminas_;
00031         double galutinisMed_;
00032         double galutinisVid_;
00033         double pazymiuVidurkis_;
00034         vector<int> pazymiai_;
00035
00036     public:
00037         Studentas(string var, string pav, vector<int> paz, int egz) :
00038             Zmogus(var, pav), pazymiai_{paz}, egzaminas_{egz} {
00039             skaiciuotiGalutiniSuMed();
00040             skaiciuotiGalutiniSuVid();
00041         }
00042
00043         Studentas(string var, string pav) : Zmogus(var, pav), egzaminas_(0) {
00044         }
00045         Studentas() : Zmogus(), egzaminas_(0) {}
00046         Studentas(std::istream& is);
00047
00048         //copy konstruktorius
00049         Studentas(const Studentas &s)
00050             : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
00051             galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
00052             pazymiuVidurkis_{s.pazymiuVidurkis_},
00053             pazymiai_{s.pazymiai_}{}
00054
00055         //move konstruktorius
00056         Studentas(Studentas&& s)
00057             : Zmogus(s.vardas(), s.pavarde()), egzaminas_{s.egzaminas_},
00058             galutinisMed_{s.galutinisMed_}, galutinisVid_{s.galutinisVid_},
00059             pazymiuVidurkis_{s.pazymiuVidurkis_},
00060             pazymiai_{std::move(s.pazymiai_)}{}
00061
00062         //destruktorius
00063         ~Studentas() override{
00064             pazymiai_.clear();
00065         };
00066
00067         //get'eriai
00068         inline double galutinisMed() const { return galutinisMed_; }
00069         inline double galutinisVid() const { return galutinisVid_; }
00070         inline const vector<int>& pazymiai() const { return pazymiai_; }
00071         inline int egzaminas() const { return egzaminas_; }
00072
00073         //set'eriai
00074         inline void setGalutinisV(double galutVid) { galutinisVid_ = galutVid; }
00075         inline void setGalutinisM(double galutMed) { galutinisMed_ = galutMed; }
00076         inline void setEgzaminas(int egz) { egzaminas_ = egz; }
00077         inline void setPazymiai(vector<int> paz) { pazymiai_ = paz; }
00078
00079         //member funkcijos
00080         void skaiciuotiGalutiniSuVid();
00081         void skaiciuotiGalutiniSuMed();
00082
00083         //copy assignment operatorius
00084         Studentas& operator=(const Studentas& s){
00085             if (this == &s) return *this;
00086
00087             setVardas(s.vardas());
00088             setPavarde(s.pavarde());
00089             egzaminas_ = s.egzaminas_;
00090             galutinisMed_ = s.galutinisMed_;
00091             galutinisVid_ = s.galutinisVid_;
00092             pazymiuVidurkis_ = s.pazymiuVidurkis_;
00093             pazymiai_ = s.pazymiai_;
00094
00095             return *this;
00096         }
00097
00098         //move assignment operatorius
00099         Studentas& operator=(Studentas&& s){
00100             if (this == &s) return *this;
00101
00102             setVardas(s.vardas());
00103             setPavarde(s.pavarde());
00104             egzaminas_ = s.egzaminas_;
00105             galutinisMed_ = s.galutinisMed_;
00106             galutinisVid_ = s.galutinisVid_;
00107             pazymiuVidurkis_ = s.pazymiuVidurkis_;
00108             pazymiai_ = std::move(s.pazymiai_);
00109
00110             s.setVardas("");
00111             s.setPavarde("");
00112
00113             return *this;
00114         }

```

```

00114         //output operatorius
00115         friend std::ostream& operator<<(std::ostream& out, const Studentas& s);
00116
00117         //input operatorius
00118         friend std::istream& operator>>(std::istream& cin, Studentas& s);
00119     };
00120
00121     bool compare(const Studentas&, const Studentas&);
00122     bool comparePagalPavarde(const Studentas&, const Studentas&);
00123     bool comparePagalEgza(const Studentas&, const Studentas&);
00124
00125     void rodytiRezultatus(vector<Studentas> studentuSarasas);
00126     void generuotiPazymius(vector<Studentas> &studentuSarasas);
00127     void generuotiStudentus(vector<Studentas> &studentuSarasas);
00128     void generuotiFailus(int studentuSkaicius);
00129     string pasirinktiFaila();
00130     void nuskaitytiFaila(string fail, vector<Studentas> &studentuSarasas);
00131     void rodytiVisusRezultatus(vector<Studentas> studentuSarasas);
00132     void testuotiFailuNuskaityma(vector<Studentas> studentuSarasas, int kartai);
00133
00134     //rikiavimo funkcijos
00135     void rikiuotiPagalVarda(vector<Studentas> &studentuSarasas);
00136     void rikiuotiPagalPavarde(vector<Studentas> &studentuSarasas);
00137     void rikiuotiPagalGalutiniMed(vector<Studentas> &studentuSarasas);
00138     void rikiuotiPagalGalutiniVid(vector<Studentas> &studentuSarasas);
00139
00140     void pasirinktiRikiavima(vector<Studentas> studentuSarasas);
00141     void skirstytiStudentus(vector<Studentas> &studentuSarasas);
00142
00143     bool vardoTikrinimas(const string& vard);
00144     void isvestiDuFailus(vector<Studentas> grupe1, vector<Studentas> grupe2);
00145     void rikiavimasIrIrasymasVargsiukamsIrKietekams(vector<Studentas> vargsiukai, vector<Studentas>
        kietekai);
00146     void testuotiDestruktoriu();
00147
00148 #endif

```

6.3 timer.h

```

00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00006 class Timer {
00007     private:
00008         std::chrono::time_point<std::chrono::high_resolution_clock> start;
00009     public:
00010         Timer() : start{std::chrono::high_resolution_clock::now()} {}
00011         void reset() {
00012             start = std::chrono::high_resolution_clock::now();
00013         }
00014         double elapsed() const {return
            std::chrono::duration<double>(std::chrono::high_resolution_clock::now() - start).count();
00015         }
00016 };
00017
00018 #endif

```


Index

README, [1](#)

Studentas, [11](#)

Timer, [12](#)

Zmogus, [12](#)